
From DDPM to Rectified Flow: a Measured, From-Scratch Diffusion Study

Yi Hou

University of Chinese Academy of Sciences
Beijing, China
houyi25@mailsucas.ac.cn

Abstract

This report documents a from-scratch implementation and measurement study of the modern diffusion stack. No `diffusers`, `flash_attn` or pretrained generative component is used: DDPM, DDIM, a diffusion transformer (DiT) with adaLN-Zero conditioning, and rectified flow are all implemented in this repository, together with the training pipeline, the sampler suite and the evaluation code. The study is organized as a controlled 2×2 design (UNet vs. DiT $\times$ ϵ -prediction vs. rectified flow) at two scales, MNIST 28×28 and CelebA 64×64 , with all cells trained under one frozen protocol. Headline numbers: TODO (FID at matched NFE for DDPM/DDIM/rectified flow; the sampler that reaches the same FID with the fewest function evaluations and its measured samples-per-second; the step-time effect of mixed precision and compilation; and a reverse benchmark showing where sharding stops paying off at this model scale).

1 Introduction

Diffusion models are usually consumed through a library: a scheduler object, a pretrained backbone, a pipeline. This report takes the opposite route. Everything is built from scratch — the forward and reverse processes, the two backbones, both training objectives, the samplers and the evaluation — and every claim is backed by a measurement produced by this repository.

The study asks three questions:

- **RQ1 (architecture).** At a similar parameter scale, does replacing a UNet with a transformer (DiT) backbone help or hurt generation quality and computational efficiency?
- **RQ2 (objective and sampler).** How does a rectified-flow objective change the quality–NFE trade-off, and how much does sampler choice alone buy for a fixed trained model (DDPM ancestral vs. DDIM)?
- **RQ3 (systems).** How much training and inference throughput can be recovered by mixed precision, compilation and kernel-level fusion at this scale — and where does a distributed strategy stop being worth it?

Section 2 puts the three objectives on a common footing. Section 3 describes the implementation, Section 4 the frozen protocol and the evaluation protocol, Sections 5–7 the results, and Section 8 collects the experiments that failed, which were as instructive as the ones that worked.

Table 1: Results at a glance. Values marked – are still being measured; the raw archives live in `results/*.json`.

Question	Baseline	Measured effect
RQ1 backbone	UNet (174.1M, 67.8 GFLOPs/sample)	DiT (129.6M, 43.6 GFLOPs/sample): FID – vs. –; step time –
RQ2 objective	ϵ -prediction	rectified flow: FID – vs. – at equal NFE
RQ2 sampler	DDPM ancestral (1000 NFE)	DDIM: FID – at – NFE for the same weights
RQ3 systems	FP32, uncompiled	BF16 –×; <code>torch.compile</code> –×; MFU –%
RQ3 scaling	DDP	FSDP reverse benchmark: –

Table 2: The three objectives on a common footing. $\bar{\alpha}_t$ is the usual DDPM cumulative product; the flow variant uses a continuous $t \in [0, 1]$.

Method	Path x_t	Target	Sampler (NFE)
DDPM [1]	$\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$	ϵ	ancestral, stochastic (1000)
DDIM [2]	same	ϵ	deterministic ODE (10–50)
Rectified flow [8, 9]	$(1 - t) x_0 + t \epsilon$	$v = \epsilon - x_0$	Euler ODE (4–128)

2 Background: one path, three objectives

All three methods place the data and the noise at the ends of an interpolation path and train a network to regress a target along it. They differ in the path, in the target, and in the solver used to walk back from noise to data.

One implementation detail is easy to get wrong and worth stating: the sinusoidal timestep embedding used by both backbones is calibrated for $t \in [0, 1000]$ (the DDPM range). The flow code therefore keeps its path mathematics in $t \in [0, 1]$ but passes $t \times 1000$ to the network; without that scaling the embedding is nearly constant and conditioning collapses.

3 Implementation

Everything is written in PyTorch, with no generative-model library. Table 3 lists the measured sizes. Note that the parameter ranking and the FLOPs ranking disagree across scales — on CelebA the UNet is larger on both axes, on MNIST it has fewer parameters but more FLOPs — which is exactly why both are reported.

Backbones. The UNet is a residual U-net with GroupNorm, SiLU, sinusoidal timestep injection into every residual block, and `scaled_dot_product_attention` at the bottleneck. The DiT follows (author?) [7]: patchify with a strided convolution, fixed two-dimensional sincos positional embeddings, and blocks whose attention and MLP are modulated by an adaLN-Zero MLP that is zero-initialized, so every block starts as the identity (verified in `tests/test_sanity.py`: the untrained model outputs exactly zero). Attention uses PyTorch SDPA in both backbones; no fused-attention library is used.

Training. A single training loop drives both objectives. It supports `torchrun` DDP, YAML configs, checkpoint resume, an exponential moving average of the weights (all reported FID uses EMA weights), per-epoch EMA-only snapshots for quality-vs-progress curves, and cost accounting: parameters, measured FLOPs, step time, throughput, peak memory and MFU are written to `results/<run>.json` on every run.

Samplers. Ancestral DDPM sampling, deterministic DDIM sampling with a configurable NFE, and Euler integration of the flow ODE. All three reuse the same trained ϵ - or velocity-predicting network, so they can be compared at fixed weights.

4 Experimental setup

Hardware. TODO: fill in the machine actually used (GPU model, interconnect, NUMA layout). The topology matters: on the six-A6000 node used for the systems numbers, GPUs 0–3 and 4–5 sit in

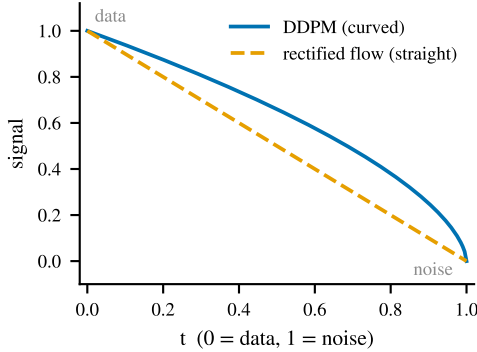


Figure 1: The diffusion path and the rectified-flow path between data and noise. The flow objective regresses a constant velocity along a straight line; the diffusion objective traverses a curved, non-uniform-speed path.

Table 3: Measured sizes (forward pass, one sample at the model’s native resolution). Params and FLOPs are counted by `eval/cost.py` with `torch.utils.flop_counter`, so both backbones are measured with the same tool.

Dataset	Backbone	Params	GFLOPs/sample	Conditioning	Attention
MNIST 28 ²	UNet	11.7M	2.43	sinusoidal + MLP	bottleneck
MNIST 28 ²	DiT	16.5M	1.05	adaLN-Zero	all 6 blocks (49 tokens)
CelebA 64 ²	UNet	174.1M	67.8	sinusoidal + MLP	bottleneck
CelebA 64 ²	DiT	129.6M	43.6	adaLN-Zero	all 12 blocks (256 tokens)

different NUMA domains and the cross-domain link is an order of magnitude slower, which is why `NCCL_P2P_DISABLE=1` is required and why DDP is kept inside one domain.

Protocol. All cells of a scale share one frozen protocol: identical optimizer, learning-rate schedule, batch size (global), EMA decay and seed (Table 4). Unlike recipes that tune each cell separately, this design means a difference between cells is attributable to the backbone or the objective rather than to the optimizer.

FID protocol. Reference statistics are computed from the training tensor itself (`data/celeba_64_uint8.pt` for CelebA, the MNIST training split for MNIST) with the same preprocessing chain used to evaluate samples; third-party statistics are never used, because any difference in resizing makes the numbers incomparable. CelebA uses the standard Inception-v3 pool3 features at 299². Inception features are not meaningful for grayscale digits, so the MNIST cells are evaluated in the feature space of a small convolutional classifier trained on MNIST inside this repository (Table 4 footnote); that metric is *non-standard* and only comparable within this report. Tier 1 evaluations use 10k samples, Tier 2 (final) 50k, and the spread is reported as the variability across five random 10k subsets.

5 Sampler study (RQ2)

TODO: table of (sampler, NFE, FID, samples/s, projected wall-clock for 50k samples) plus the prose reading of Fig. 2. Fill from `results/fid_*.json` (`eval/make_table.py` prints the table).

6 Backbone and objective (RQ1)

TODO: prose. State explicitly that parameter count is not compute: report the FLOPs column from Table 3 alongside every quality comparison, and note that the CelebA cells are the expensive ones (measured training cost in Table 6).

Table 4: The frozen protocol. Any deviation is recorded in the run’s JSON.

	MNIST 28 ²	CelebA 64 ²
Cells	UNet/DiT $\times \epsilon/\text{flow}$, one run each	
Epochs	200	300
Global batch	256	800 (160 per GPU $\times$ 5)
Optimizer	Adam, lr 2×10^{-4} , warmup then cosine to 10^{-6}	
Gradient clipping	1.0	
EMA decay	0.9999, applied in every cell	
Seed	42	

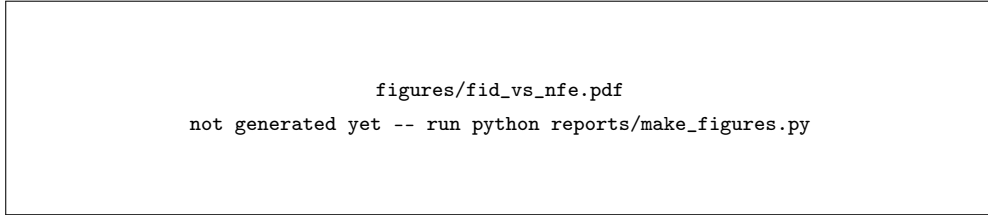


Figure 2: Quality as a function of the number of function evaluations, across generative trajectories. DDPM vs. DDIM is a same-weights sampler ablation; comparisons across cells additionally change the training objective.

7 Systems (RQ3)

TODO: the profile-first argument — which category dominated, what optimization it implied, and what the optimization actually returned. Then the distributed measurement: DDP step time and the NCCL share, and the FSDP reverse benchmark at this parameter scale (expected to be slower; the point is to measure and report it). Note the MFU caveat: the counter covers matmul and convolution work only, so "MFU" here is a matmul-utilisation proxy.

8 What didn’t work

Each entry follows the same shape: hypothesis, setup, observation, diagnosis, resolution, lesson.

Learning rate 5×10^{-4} collapses CelebA training. TODO: fill in with the observed loss/grad-norm curves; resolution was 2×10^{-4} with gradient clipping at 1.0.

DataParallel deadlocks with a multi-worker DataLoader. TODO: describe the hang and the migration to torchrun-native DDP.

P2P hangs on this node’s NUMA layout. TODO: NCCL_P2P_DISABLE=1, and what the topology measurement showed.

FSDP at this scale. TODO: reverse benchmark numbers (expected: communication overhead without meaningful memory savings at ~ 130 – 174 M parameters).

9 Discussion and limitations

- Unconditional generation only: no classifier-free guidance, no text conditioning.
- Two datasets, one resolution each; the CelebA 2×2 is the expensive half and its status is stated explicitly in each table.
- FID is computed against reference statistics built from this repository’s own preprocessing chain. Absolute values are therefore *not* comparable with published numbers; only comparisons within this report are meaningful.
- The MNIST metric is a non-standard feature-space distance.
- One seed per cell; the reported spread is FID variability across data subsets, not across seeds.
- FLOPs counts exclude elementwise work, so MFU figures are a proxy.

```

figures/throughput_vs_nfe.pdf
not generated yet -- run python reports/make_figures.py

```

Figure 3: Sampling throughput against NFE, measured on the same runs.

Table 5: The 2×2 design. FID at each cell’s best NFE; see the text for how the NFE was chosen.

Dataset	Cell	FID	NFE at that FID
MNIST	A: UNet + ϵ	-	-
MNIST	B: DiT + ϵ	-	-
MNIST	C: UNet + flow	-	-
MNIST	D: DiT + flow	-	-
CelebA	A: UNet + ϵ	-	-
CelebA	B: DiT + ϵ	-	-
CelebA	C: UNet + flow	-	-
CelebA	D: DiT + flow	-	-

Reproduction

All commands (environment, data, smoke tests, launching cells, evaluation) are in `reports/runbook.md`; the raw archive behind every number is `results/*.json`; the committed code contains the full training stack and the test suite used to validate the implementations (`tests/test_sanity.py`, 14 CPU-only invariants).

References

- [1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *NeurIPS*, 2020.
- [2] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *ICLR*, 2021.
- [3] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *ICLR*, 2021.
- [4] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. *ICML*, 2021.
- [5] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv:2207.12598*, 2022.
- [6] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. *CVPR*, 2022.
- [7] William Peebles and Saining Xie. Scalable diffusion models with transformers. *ICCV*, 2023.
- [8] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: learning to generate and transfer data with rectified flow. *ICLR*, 2023.
- [9] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. *ICLR*, 2023.
- [10] Patrick Esser et al. Scaling rectified flow transformers for high-resolution image synthesis. *ICML*, 2024.



Figure 4: EMA samples from each cell’s final checkpoint.

Table 6: Training step cost under each execution setting (30 steps, 5 warmups discarded). Produced by `eval/systems.py`; the FID column is the check that a speedup did not silently cost quality.

Config	Setting	ms/step	img/s	peak GB	MFU
TODO	FP32				
TODO	BF16				
TODO	compile				
TODO	BF16 + compile				

- [11] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. *NeurIPS*, 2017.
- [12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 2022.
- [13] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: memory optimizations toward training trillion parameter models. *SC20*, 2020.

```
figures/systems_settings.pdf
not generated yet -- run python reports/make_figures.py
```

Figure 5: Step time by execution setting.

```
figures/profile_breakdown.pdf
not generated yet -- run python reports/make_figures.py
```

Figure 6: Where GPU time goes in a diffusion training step, by kernel category. The measurement is a matmul/conv accounting only: elementwise-heavy operations (GroupNorm, SiLU, adaLN modulation) are exactly the ones a per-kernel profile reveals and a FLOPs count hides.

- [14] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. PyTorch FSDP: experiences on scaling fully sharded data parallel. *VLDB*, 2023.